FULL-STACK PROJECT MANAGEMENT PLATFORM

Project Hub

A production-grade Jira clone with Kanban boards, sprint management, real-time collaboration, and powerful reporting.

Kanban Board · Sprint Management · Real-time Updates · Reports & Charts · Team Collaboration · Activity Tracking

React 19 · FastAPI · WebSocket · TypeScript · April 2026

Meet the Team

The developers behind the Jira Clone

#01

Him Hey

Team Member

#02

Ramekh CHHOENG

Team Member

#03

Hok Hieb

Team Member

#04

Boeurn Mnor

Team Member

#05

Lay Sophanha

Team Member

#06

Doem Khouch

Team Member

Agenda

What we'll cover today

Project Overview
What we built, scope, and the full feature set

Dashboard & Reports
Summary view, burndown charts, velocity & cumulative flow

Sprint & Backlog
Sprint planning, backlog grooming, and task prioritization

Architecture
Frontend & backend design, API layer, real-time WebSocket

Real-time & UX
WebSocket sync, optimistic UI, keyboard shortcuts, search

Demo & Highlights
Live walkthrough and project achievements

Project Overview

A Jira-style project management clone

🎯 Goal

Build a production-grade project management platform that teams can actually use — with Kanban boards, sprint management, real-time collaboration, and comprehensive reporting.

✨ Key Features

- **Kanban board** with drag & drop workflow
- **Sprint management** — plan, start, complete
- **Epics & Timeline** — Gantt-style roadmap
- **Real-time sync** via WebSocket
- **Reports** — burndown, velocity, cumulative flow
- **Team & notifications** — roles, activity feed

🛠 Tech Stack

React 19	UI framework
TypeScript	Type safety
FastAPI	Backend API
React Query	Server state
WebSocket	Real-time sync
Recharts	Data visualization

20+
Pages & views

30+
API endpoints

Dashboard & Reports

Data-driven insights for project tracking

📊 Project Summary

A personalized dashboard showing task distribution by status (doughnut chart), priority breakdown (bar chart), recent activity, and quick stats cards.

📈 Reports Suite

- **Burndown Chart** — track sprint progress over time
- **Velocity Chart** — story points per sprint
- **Cumulative Flow** — visualize workflow bottlenecks
- **Status Distribution** — pie chart of task states
- **Priority & Type** — bar charts for workload analysis

🏠 For You Page

Personalized home view with assigned tasks, recent projects, starred items, and notification summary.

📋 Activity Feed

- **Full audit trail** — every create, update, delete logged
- **Field-level tracking** — old value → new value
- **Filterable** — by action type, user, date range
- **Entity linking** — click to jump to task/project

🔔 Notifications

- **Task assignment** notifications
- **Comment mentions** & replies
- **Sprint events** — started, completed
- **Mark read/unread** with categories

Sprint & Backlog

Full agile workflow — from backlog grooming to sprint completion

🏃 Sprint Management

- **Create sprints** with name, goal, and date range
- **Start sprint** — sets active status, enables board
- **Complete sprint** — move unfinished tasks to backlog or next sprint
- **Sprint history** — track velocity across completed sprints
- **Story points** — per-task estimation for capacity planning

⚡ Epic & Timeline

- **Epic management** — group related tasks under epics
- **Progress tracking** — visual progress bars per epic
- **Gantt timeline** — zoom by weeks, months, quarters
- **Pan navigation** — scroll through project timeline
- **Sprint grouping** — see tasks organized within sprints

📋 Backlog Features

- **Sprint sections** — drag tasks between sprints
- **Inline task creation** — quick-add without leaving the view
- **Filters** — by type, priority, assignee, label
- **Bulk actions** — multi-select to move, delete, or update

🔄 Sprint Workflow

📋 Plan → 🏃 Active → ✅ Complete

Unfinished tasks are automatically moved to the next sprint or back to the backlog when a sprint completes.

Component Architecture

How we structured our React application

🖥️ Pages (Routes) — Board, Backlog, Projects, Issues

↕

🧩 Feature Components — BoardColumn, IssueCard, Filters

↕

🔧 Shared UI — Button, Modal, Badge, Input, Select

↕

🪝 Custom Hooks — useProjects, useIssues, useFilters

↕

📡 Services — API client, fetch wrappers, JWT auth

📁 Folder Structure

```
src/
├── features/
│   ├── board/          ← KanbanBoard, Column, Card
│   ├── backlog/        ← BacklogList, Filters
│   ├── projects/       ← ProjectList, ProjectForm
│   └── issues/         ← IssueDetail, IssueForm
├── components/
│   ├── ui/             ← Button, Modal, Input
│   └── layout/         ← Sidebar, Header
├── hooks/            ← Custom React hooks
├── services/         ← API layer
├── store/            ← Zustand global state
└── types/            ← TypeScript interfaces
```

React Router v7

Client-side routing with nested layouts

🗺️ Route Structure

- **/** — Dashboard overview
- **/board** — Kanban board view
- **/backlog** — Backlog management
- **/projects** — Project list
- **/projects/:id** — Project detail
- **/issues/:key** — Issue detail drawer

🔒 Protected Routes

Routes are wrapped in an **AuthLayout** component that checks for a valid JWT token. Unauthenticated users are redirected to login.

📝 Route Configuration

```
const router = createBrowserRouter([
  {
    path: '/',
    element: <AuthLayout />,
    children: [
      {
        element: <DashboardLayout />,
        children: [
          { index: true, element: <Dashboard /> },
          { path: 'board', element: <Board /> },
          { path: 'backlog', element: <Backlog /> },
          { path: 'projects', element: <Projects /> },
          {
            path: 'projects/:projectId',
            element: <ProjectDetail />
          },
        ]
      }
    ]
  },
  { path: '/login', element: <Login /> },
]);
```

State Management

Separating server state from client state

🔄 React Query — Server State

- **useQuery** — fetch & cache server data
- **useMutation** — create, update, delete
- **Optimistic updates** — instant UI feedback
- **Auto refetch** — keep data fresh
- **Cache invalidation** — sync after mutations

🐻 Zustand — Client State

- **UI state** — sidebar open/closed, theme
- **Auth state** — current user, token
- **Filter state** — board/backlog selections

📝 React Query Pattern

```
// useIssues.ts
export function useIssues(projectId?: string) {
  return useQuery({
    queryKey: ['issues', projectId],
    queryFn: () => fetchIssues(projectId),
    staleTime: 30_000,
  });
}

// useUpdateIssue.ts
export function useUpdateIssue() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: updateIssue,
    onSuccess: () => {
      queryClient.invalidateQueries({
        queryKey: ['issues']
      });
    },
  });
}
```

Kanban Board & Drag Drop

Interactive board powered by @dnd-kit

📋 Board Columns

TO DO | IN PROGRESS | IN REVIEW | DONE

Issues are grouped by workflow status. Dragging an issue between columns triggers a **PATCH /issues/:key** API call and React Query cache update.

🔎 Board Filters

- **Project** — multi-project view
- **Issue Type** — story, bug, task, epic
- **Assignee** — filter by person
- **Search** — text search across issues

🧲 Drag & Drop Code

```
import {
  DndContext, DragOverlay,
  useSensor, PointerSensor
} from '@dnd-kit/core';

function KanbanBoard() {
  const sensors = useSensors(
    useSensor(PointerSensor, {
      activationConstraint: { distance: 8 }
    })
  );

  const handleDragEnd = ({ active, over }) => {
    if (!over || active.id === over.id) return;
    updateIssue.mutate({
      key: active.id,
      status: over.id  // new column status
    });
  };

  return (
    <DndContext
      sensors={sensors}
      onDragEnd={handleDragEnd}
    >
      {columns.map(col =>
        <BoardColumn key={col.id} column={col} />
      )}
    </DndContext>
  );
}
```

TypeScript Integration

Type-safe development for a better developer experience

✅ Benefits

- **Catch errors at compile time** — not in production
- **Auto-complete** — IDE knows all prop types
- **Refactor safely** — TypeScript flags all breakages
- **Self-documenting** — types serve as documentation
- **API contracts** — typed server responses

🏷️ Type Coverage

Props | API Responses | Events | Store | Route Params

📦 Core Types

```typescript
type IssueType = 'story' | 'task' | 'bug' | 'epic';
type Priority = 'highest' | 'high' | 'medium' | 'low';
type IssueStatus = 'todo' | 'in_progress' | 'in_review' | 'done';

interface Issue {
  id: string;
  key: string;           // e.g. "WEB-42"
  title: string;
  description?: string;
  type: IssueType;
  status: IssueStatus;
  priority: Priority;
  projectId: string;
  reporter: string;
  assignee?: string;
  dueDate?: string;
  createdAt: string;
  updatedAt: string;
}

interface Project {
  id: string;
  key: string;           // e.g. "WEB"
  name: string;
  description: string;
  status: string;
  priority: Priority;
  startDate: string;
  endDate?: string;
  tags: string[];
}
```

Forms & Validation

React Hook Form + Zod schema validation

📝 React Hook Form

- **Uncontrolled inputs** — minimal re-renders
- **register()** — connects inputs to form
- **handleSubmit()** — validates before submit
- **formState.errors** — per-field error messages
- **watch()** — observe field value changes

⚡ Zod Validation

- **Schema-first** — define shape & rules
- **Type inference** — auto TypeScript type
- **Custom messages** — user-friendly errors

📝 Issue Form Example

```
const issueSchema = z.object({
  title: z.string().min(3, 'Title too short'),
  type: z.enum(['story','task','bug','epic']),
  priority: z.enum(['highest','high','medium','low']),
  assignee: z.string().optional(),
  dueDate: z.string().optional(),
});

type IssueFormData = z.infer<typeof issueSchema>;

function IssueForm() {
  const { register, handleSubmit, formState }
    = useForm<IssueFormData>({
      resolver: zodResolver(issueSchema)
    });

  const onSubmit = (data: IssueFormData) => {
    createIssue.mutate(data);
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('title')} />
      {formState.errors.title && (
        <p>{formState.errors.title.message}</p>
      )}
    </form>
  );
}
```

Issue Model — Jira Style

Rich issue metadata inspired by Jira's data model

🏷️ Issue Types

📖 Story | ✅ Task | 🐞 Bug | ⚡ Epic

Each issue gets an auto-generated key like **WEB-1**, **API-12** — sequential per project.

⚡ Priority Levels

🔴 Highest | 🟠 High | 🟡 Medium | 🔵 Low

🔍 Search Scope

- **Issue key** (WEB-1)
- **Title & description**
- **Reporter & assignee**
- **Project name & key**

📊 Issue Workflow

📋 TODO — Issue created, ready to start

↓

⚙️ IN PROGRESS — Developer is working on it

↓

👁️ IN REVIEW — Code review / QA stage

↓

✅ DONE — Completed and verified

Status changes happen via **drag & drop** on the board or **inline select** in the backlog and issue detail drawer.

Backend & API

Python FastAPI backend with 30+ REST endpoints & WebSocket

METHOD	ENDPOINT	DESCRIPTION
GET	/projects	List all projects
POST	/projects	Create project
GET	/issues	List/filter issues
POST	/issues	Create issue
PATCH	/issues/:key	Update issue
DELETE	/issues/:key	Delete issue

🔑 Auth — JWT

POST /login → JWT Token → Authorization header

📡 API Service Layer

```
// services/api.ts
const api = {
  get: async<T>(path: string): Promise<T> => {
    const res = await fetch(`${BASE_URL}${path}`, {
      headers: {
        'Authorization': `Bearer ${getToken()}`,
        'Content-Type': 'application/json',
      },
    });
    if (!res.ok) throw new ApiError(res.status);
    return res.json();
  },

  patch: async<T>(path: string, body: unknown) => {
    const res = await fetch(`${BASE_URL}${path}`, {
      method: 'PATCH',
      body: JSON.stringify(body),
      headers: {
        'Authorization': `Bearer ${getToken()}`,
        'Content-Type': 'application/json',
      },
    });
    return res.json() as T;
  },
};
```

Project Achievements

What we delivered and how we built it

🚀

Production-Ready

Docker deployment, JWT security, PostgreSQL support, and progressive web app (PWA) ready

⚡

Real-time Sync

WebSocket integration with auto-reconnect so all team members see updates instantly

📊

Data Visualization

Burndown, velocity, and cumulative flow charts that give real project insights

🧩

Full-Stack Ownership

Built both FastAPI backend (30+ endpoints) and React frontend (20+ pages) end-to-end

🎯

Complete Agile Workflow

Epics, sprints, backlog, Kanban, timeline — a full project management lifecycle

🤝

Team Collaboration

Role-based access, task assignments, comments, notifications, and activity tracking

🎉

Thank You!

We built a production-grade project management platform from the ground up — ready for real teams to use.

Him Hey · Ramekh CHHOENG · Hok Hieb · Boeurn Mnor · Lay Sophanha · Doem Khouch

Built with

React 19 · TypeScript · FastAPI · React Query · WebSocket · Recharts · @hello-pangea/dnd · Radix UI

Questions? Let's discuss! 💬